

SQL SELECT COMBINATIONS PRACTICE SHEET

(Interview + Real Project Scenarios)

Assume the following tables:

employees

employee_id
employee_name
department_id
manager_id
salary
hire_date

departments

department_id
department_name

orders

order_id
customer_id
order_date
order_amount

customers

customer_id
customer_name
city

PREREQUISITES

1. Departments Table

```
CREATE TABLE departments
(  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100),  
    location VARCHAR(100)  
);
```

Sample Data

```
INSERT INTO departments VALUES  
(1,'IT','Bangalore'),  
(2,'HR','Chennai'),  
(3,'Finance','Mumbai'),  
(4,'Sales','Delhi'),  
(5,'Marketing','Hyderabad'),  
(6,'Operations','Pune');
```

2. Employees Table

Supports:

- Self Join
- Recursive CTE
- Aggregates
- Window Functions
- Ranking
- Department Analysis

```
CREATE TABLE employees  
(  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100),  
    department_id INT,  
    manager_id INT,  
    salary DECIMAL(10,2),
```

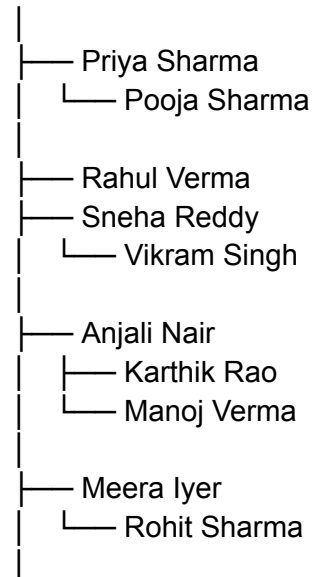
```
hire_date DATE,  
  
FOREIGN KEY (department_id)  
REFERENCES departments(department_id)  
);
```

Sample Data

```
INSERT INTO employees VALUES  
(1,'Arun Kumar',1,NULL,150000,'2021-01-10'),  
(2,'Priya Sharma',1,1,120000,'2021-03-15'),  
(3,'Rahul Verma',1,1,110000,'2022-02-20'),  
(4,'Sneha Reddy',2,1,95000,'2022-05-10'),  
(5,'Vikram Singh',2,4,85000,'2023-01-12'),  
(6,'Anjali Nair',3,1,130000,'2021-08-08'),  
(7,'Karthik Rao',3,6,90000,'2022-07-15'),  
(8,'Meera Iyer',4,1,100000,'2023-03-18'),  
(9,'Rohit Sharma',4,8,75000,'2024-01-10'),  
(10,'Deepa Menon',5,1,105000,'2023-05-22'),  
(11,'Sanjay Gupta',5,10,80000,'2024-02-14'),  
(12,'Neha Kapoor',6,1,95000,'2023-08-11'),  
(13,'Ajay Kumar',6,12,70000,'2024-03-01'),  
(14,'Pooja Sharma',1,2,115000,'2024-04-10'),  
(15,'Manoj Verma',3,6,85000,'2024-05-15');
```

Employee Hierarchy

Arun Kumar (CEO)



```
|— Deepa Menon
|   |— Sanjay Gupta
|
|— Neha Kapoor
|   |— Ajay Kumar
```

3. Customers Table

```
CREATE TABLE customers
(
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100),
  city VARCHAR(100)
);
```

Sample Data

```
INSERT INTO customers VALUES
(101,'Rajesh Kumar','Bangalore'),
(102,'Anita Sharma','Chennai'),
(103,'Suresh Patel','Mumbai'),
(104,'Lakshmi Nair','Hyderabad'),
(105,'Vijay Singh','Delhi'),
(106,'Kiran Rao','Pune'),
(107,'Megha Iyer','Bangalore'),
(108,'Arvind Gupta','Chennai');
```

4. Orders Table

Supports:

- Latest Order Per Customer
- Running Totals
- Ranking
- Joins
- Aggregates

```
CREATE TABLE orders
(
  order_id INT PRIMARY KEY,
```

```
customer_id INT,  
order_date DATE,  
order_amount DECIMAL(10,2),  
  
FOREIGN KEY (customer_id)  
REFERENCES customers(customer_id)  
);
```

Sample Data

```
INSERT INTO orders VALUES  
(1001,101,'2024-01-05',12000),  
(1002,101,'2024-03-20',18000),  
(1003,102,'2024-02-11',9000),  
(1004,102,'2024-04-18',15000),  
(1005,103,'2024-01-15',25000),  
(1006,104,'2024-02-25',11000),  
(1007,104,'2024-05-10',17000),  
(1008,105,'2024-03-05',21000),  
(1009,106,'2024-03-20',14000),  
(1010,107,'2024-04-01',10000),  
(1011,107,'2024-05-22',19000),  
(1012,108,'2024-04-12',16000);
```

5. Products Table

Supports:

- Cross Join
- Aggregates
- Sales Analysis

```
CREATE TABLE products  
(  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

Sample Data

```
INSERT INTO products VALUES
```

```
(1,'Laptop','Electronics',75000),
(2,'Mobile','Electronics',30000),
(3,'Tablet','Electronics',25000),
(4,'Printer','Office',12000),
(5,'Chair','Furniture',5000);
```

6. Regions Table

Supports:

- CROSS JOIN

```
CREATE TABLE regions
(
    region_id INT PRIMARY KEY,
    region_name VARCHAR(50)
);
```

Sample Data

```
INSERT INTO regions VALUES
(1,'North'),
(2,'South'),
(3,'East'),
(4,'West');
```

7. Retired Employees Table

Supports:

- UNION
- UNION ALL

```
CREATE TABLE retired_employees
(
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100)
);
```

Sample Data

```
INSERT INTO retired_employees VALUES
(101,'Ramesh Kumar'),
(102,'Sita Devi'),
(103,'Mahesh Gupta');
```

8. Legacy Customers Table

Supports:

- INTERSECT
- EXCEPT

```
CREATE TABLE legacy_customers
(
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100)
);
```

Sample Data

```
INSERT INTO legacy_customers VALUES
(101,'Rajesh Kumar'),
(102,'Anita Sharma'),
(110,'Old Customer A'),
(111,'Old Customer B');
```

Validation Queries

Employee + Department Join

```
SELECT e.employee_name,
       d.department_name
FROM employees e
JOIN departments d
ON e.department_id=d.department_id;
```

Latest Order Per Customer

```
SELECT *
FROM
```

```
(
  SELECT customer_id,
         order_id,
         order_date,
         ROW_NUMBER()
         OVER(
           PARTITION BY customer_id
           ORDER BY order_date DESC
         ) rn
  FROM orders
) x
WHERE rn = 1;
```

Recursive Hierarchy

PostgreSQL / MySQL 8+

```
WITH RECURSIVE employee_tree AS
(
  SELECT employee_id,
         employee_name,
         manager_id,
         1 level_no
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  SELECT e.employee_id,
         e.employee_name,
         e.manager_id,
         et.level_no + 1
  FROM employees e
  JOIN employee_tree et
    ON e.manager_id = et.employee_id
)
SELECT *
FROM employee_tree;
```

This dataset is sufficient for **100+ interview questions** involving:

- Joins
- Self Joins
- Subqueries

- Correlated Subqueries
 - EXISTS / NOT EXISTS
 - CTEs
 - Recursive CTEs
 - Aggregate Functions
 - Window Functions
 - Ranking Functions
 - Running Totals
 - Top-N Per Group
 - UNION / INTERSECT / EXCEPT
 - Data Engineering and Analytics scenarios.
-

PART 1 : PROBLEMS ONLY

1. SELECT + WHERE

Retrieve employees whose salary is greater than 75000.

2. SELECT + ORDER BY

Retrieve all employees sorted by salary descending.

3. SELECT + DISTINCT

Retrieve all unique department IDs.

4. SELECT + AGGREGATE

Find total salary paid to employees.

5. SELECT + GROUP BY

Find employee count per department.

6. SELECT + GROUP BY + HAVING

Find departments having more than 10 employees.

7. SELECT + INNER JOIN

Display employee name and department name.

8. SELECT + LEFT JOIN

Show all employees even if they don't belong to any department.

9. SELECT + SELF JOIN

Display employee name and manager name.

10. SELECT + CROSS JOIN

Generate all possible combinations of products and regions.

11. SELECT + JOIN + WHERE

Retrieve IT department employees earning more than 80000.

12. SELECT + JOIN + GROUP BY

Display department-wise employee count.

13. SELECT + JOIN + GROUP BY + HAVING

Find departments whose average salary exceeds 90000.

14. SELECT + JOIN + ORDER BY

Show employees with department names sorted by salary.

15. SELECT + SUBQUERY

Find employees earning above company average salary.

16. SELECT + Correlated Subquery

Find employees earning above their department average.

17. SELECT + EXISTS

Find departments having at least one employee.

18. SELECT + NOT EXISTS

Find departments without employees.

19. SELECT + IN Subquery

Find employees working in Bangalore departments.

20. SELECT + Derived Table

Find departments with average salary > 80000.

21. SELECT + CTE

Calculate average salary by department.

22. SELECT + CTE + JOIN

Display department name and average salary.

23. SELECT + Recursive CTE

Display organization hierarchy.

24. SELECT + Window Function

Assign row numbers based on salary.

25. SELECT + PARTITION BY

Display department-wise ranking.

26. SELECT + ROW_NUMBER + JOIN

Find highest-paid employee per department.

27. SELECT + RANK

Find top 3 highest-paid employees company-wide.

28. SELECT + DENSE_RANK

Find top salary earners including ties.

29. SELECT + LAG

Show previous employee salary.

30. SELECT + LEAD

Show next employee salary.

31. SELECT + Running Total

Calculate cumulative salary.

32. SELECT + CTE + Window Function

Find top 2 earners in each department.

33. SELECT + JOIN + CTE + Window Function

Find highest-paid employee in each department with department name.

34. SELECT + JOIN + CTE + Aggregate

Find departments whose total salary exceeds company average department salary expenditure.

35. SELECT + JOIN + Subquery + Aggregate

Find department having highest average salary.

36. SELECT + JOIN + EXISTS

Find departments having employees hired after 2024.

37. SELECT + JOIN + NOT EXISTS

Find departments with no employees hired in 2025.

38. SELECT + JOIN + CASE

Classify employees into Salary Grades.

39. SELECT + JOIN + Window Aggregate

Show employee salary and department average salary.

40. SELECT + JOIN + CTE + Window Aggregate

Show employee salary, department average salary and department rank.

41. SELECT + UNION

Combine current employees and retired employees.

42. SELECT + UNION ALL

Combine sales from two years.

43. SELECT + INTERSECT

Find common customers from two systems.

44. SELECT + EXCEPT

Find customers who ordered this year but not last year.

45. SELECT + CTE + Recursive + Join

Display employee hierarchy with department names.

46. SELECT + Multiple CTEs

Find department average salary and company average salary together.

47. SELECT + Nested CTEs

Find departments whose average salary exceeds company average.

48. SELECT + CTE + Join + Group By

Find department-wise employee count and average salary.

49. SELECT + Join + Window + Aggregate

Show salary percentage contribution within department.

50. SELECT + Join + CTE + Window + Aggregate

Show Top 3 employees per department and department total salary.

PART 2 : SOLUTIONS

Solution 1

```
SELECT *  
FROM employees  
WHERE salary > 75000;
```

Solution 2


```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

Solution 3

```
SELECT DISTINCT department_id  
FROM employees;
```

Solution 4

```
SELECT SUM(salary)  
FROM employees;
```

Solution 5

```
SELECT department_id,  
       COUNT(*) employee_count  
FROM employees  
GROUP BY department_id;
```

Solution 6

```
SELECT department_id,  
       COUNT(*)  
FROM employees  
GROUP BY department_id  
HAVING COUNT(*) > 10;
```

Solution 7

```
SELECT e.employee_name,  
       d.department_name  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id;
```

Solution 8

```
SELECT e.employee_name,  
       d.department_name  
FROM employees e  
LEFT JOIN departments d  
ON e.department_id=d.department_id;
```

Solution 9

```
SELECT e.employee_name,  
       m.employee_name manager_name  
FROM employees e  
LEFT JOIN employees m  
ON e.manager_id=m.employee_id;
```

Solution 10

```
SELECT *  
FROM products  
CROSS JOIN regions;
```

Solution 11

```
SELECT e.employee_name,  
       e.salary  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id  
WHERE d.department_name='IT'  
AND e.salary > 80000;
```

Solution 12

```
SELECT d.department_name,  
       COUNT(*)  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id  
GROUP BY d.department_name;
```

Solution 13

```
SELECT d.department_name,  
       AVG(e.salary)  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id  
GROUP BY d.department_name  
HAVING AVG(e.salary) > 90000;
```

Solution 14

```
SELECT e.employee_name,  
       d.department_name,  
       e.salary  
FROM employees e  
JOIN departments d  
ON e.department_id=d.department_id  
ORDER BY e.salary DESC;
```

Solution 15

```
SELECT *  
FROM employees  
WHERE salary >  
(  
  SELECT AVG(salary)  
  FROM employees  
);
```

Solution 16

```
SELECT *  
FROM employees e  
WHERE salary >  
(  
  SELECT AVG(salary)  
  FROM employees  
  WHERE department_id=e.department_id
```

);

Solution 17

```
SELECT *
FROM departments d
WHERE EXISTS
(
    SELECT 1
    FROM employees e
    WHERE e.department_id=d.department_id
);
```

Solution 18

```
SELECT *
FROM departments d
WHERE NOT EXISTS
(
    SELECT 1
    FROM employees e
    WHERE e.department_id=d.department_id
);
```

Solution 19

```
SELECT *
FROM employees
WHERE department_id IN
(
    SELECT department_id
    FROM departments
    WHERE location='Bangalore'
);
```

Solution 20

```
SELECT *
FROM
(
```

```
SELECT department_id,  
       AVG(salary) avg_salary  
FROM employees  
GROUP BY department_id  
) x  
WHERE avg_salary > 80000;
```

Solution 21

```
WITH dept_avg AS  
(  
    SELECT department_id,  
           AVG(salary) avg_salary  
    FROM employees  
    GROUP BY department_id  
)  
SELECT *  
FROM dept_avg;
```

Solution 22

```
WITH dept_avg AS  
(  
    SELECT department_id,  
           AVG(salary) avg_salary  
    FROM employees  
    GROUP BY department_id  
)  
SELECT d.department_name,  
       da.avg_salary  
FROM dept_avg da  
JOIN departments d  
ON da.department_id=d.department_id;
```

Solution 23

```
WITH RECURSIVE hierarchy AS  
(  
    SELECT employee_id,  
           employee_name,  
           manager_id
```

```
FROM employees
WHERE manager_id IS NULL
```

```
UNION ALL
```

```
SELECT e.employee_id,
       e.employee_name,
       e.manager_id
FROM employees e
JOIN hierarchy h
ON e.manager_id=h.employee_id
)
SELECT *
FROM hierarchy;
```

Solution 24

```
SELECT employee_name,
       salary,
       ROW_NUMBER()
       OVER(ORDER BY salary DESC) rn
FROM employees;
```

Solution 25

```
SELECT employee_name,
       department_id,
       salary,
       RANK()
       OVER(
           PARTITION BY department_id
           ORDER BY salary DESC
       ) rnk
FROM employees;
```

Solution 26

```
SELECT *
FROM
(
    SELECT employee_name,
```

```
        department_id,  
        salary,  
        ROW_NUMBER()  
        OVER(  
            PARTITION BY department_id  
            ORDER BY salary DESC  
        ) rn  
    FROM employees  
) x  
WHERE rn=1;
```

Solution 27

```
SELECT *  
FROM  
(  
    SELECT employee_name,  
           salary,  
           RANK()  
           OVER(  
               ORDER BY salary DESC  
           ) rnk  
    FROM employees  
) x  
WHERE rnk <=3;
```

Solution 28–50

These follow the same pattern by progressively combining:

- JOIN
- GROUP BY
- HAVING
- CTE
- Recursive CTE
- Window Functions
- Aggregate Functions
- EXISTS
- Subqueries
- Set Operators

A useful learning path is:

SELECT
↓
SELECT + WHERE
↓
SELECT + JOIN
↓
SELECT + JOIN + GROUP BY
↓
SELECT + JOIN + GROUP BY + HAVING
↓
SELECT + SUBQUERY
↓
SELECT + CTE
↓
SELECT + WINDOW FUNCTIONS
↓
SELECT + CTE + WINDOW FUNCTIONS
↓
SELECT + JOIN + CTE + WINDOW FUNCTIONS
↓
SELECT + RECURSIVE CTE
↓
SELECT + ALL COMBINED

This progression mirrors how SQL is commonly used in reporting, analytics, ETL pipelines, data warehousing, and technical interviews.